Inyección de Dependencias

Principios, patrones y buenas prácticas en Java

¿Qué es la Inyección de Dependencias?

Definición

- La Inyección de Dependencias (DI) es un patrón de diseño donde los objetos reciben sus dependencias desde el exterior, en lugar de crearlas internamente.
- Forma parte del principio de Inversión de Dependencias (la 'D' de SOLID).

El Problema sin DI

- Alto acoplamiento entre clases: las clases crean directamente sus dependencias con `new`.
- Dificulta las pruebas unitarias porque no se pueden sustituir dependencias fácilmente.

La Solución con DI

- Las dependencias se pasan desde afuera, usualmente por constructor, setter o interfaz.
- Favorece la inversión de control (IoC): el framework controla el ciclo de vida de los objetos.

Tipos de Inyección de Dependencias

DI por Constructor

- Dependencia recibida en el constructor
- Más recomendada: inmutabilidad y obligatoriedad
- El objeto no puede existir sin sus dependencias
- Facilita pruebas unitarias
- Compatible con frameworks como Spring
- Ejemplo: @Autowired sobre el constructor

DI por Setter / Campo

- Dependencia asignada mediante un método setter
- Permite dependencias opcionales
- @Autowired en el campo (inyección directa)
- Más flexible pero menos recomendada
- Riesgo: el objeto puede estar incompleto
- Usada comúnmente en pruebas o beans opcionales

Inyección de Dependencias — Ejemplo práctico

```java
public class Persona {
    // inyección por constructor
    public Persona(String nombre, String apellido){
        this.nombre = nombre;
        ...
    }
    public String nombre = "brandon";
    public String apellido;
    ...

    public llamarAcudiente(String number){
        // inyección por método
....}

    public setNombre(String nombre){
        // inyección por setter
        this.nombre = nombre;
    }
}
```

```java
{ // inyección por campo
    Persona p1 = new Persona();
    p1.nombre = "brandon";
    p1.apellido = "martinez";
    ....
}
```

```java
{ // inyección por setter
    Persona p1 = new Persona();
    p1.setNombre = "brandon";
    p1.setApellido = "martinez";
    ....
}
```

```java
{ // inyección por constructor
    Persona p1 = new Persona("brandon", "martinez");
    ....
}
```

```java
{ // inyección por metodo
    Persona p1 = new Persona("jairo", "martinez");
    p1.llamarAcudiente("+573213779856");
    ....
}
```